

variable ❶. Normally, lookup tables are hardcoded directly in the source code, from `MULT_TABLE[[0, 0]] = 0` to `MULT_TABLE[[9, 9]] = 81`. But to reduce the amount of typing, we'll use nested `for` loops to generate each product. Accessing `MULT_TABLE[[m, n]]` gives us the product of integers `m` and `n`.

Our `karatsuba()` function also relies on a helper function named `padZeros()`, which pads a string of digits with additional zeros on the left or right side of the string. This padding is done in the fifth step of the Karatsuba algorithm. For example, `padZeros("42", 3, "left")` returns the string `00042`, while `padZeros("99", 1, "right")` returns the string `990`.

The `karatsuba()` function itself first checks for the base case, where `x` and `y` are single-digit numbers. These can be multiplied using the lookup table, and their product is immediately returned. Everything else is a recursive case.

We need to convert the `x` and `y` integers into strings and adjust them so that they contain the same number of digits. If one of these numbers is shorter than the other, zeros are padded to the left side. For example, if `x` is 13 and `y` is 2468, our function calls `padZeros()` so that `x` can be replaced with `0013`. This is required because we then create the `a`, `b`, `c`, and `d` variables to each contain one-half of the digits of `x` and `y` ❷. The `a` and `c` variables must have the same number of digits for the Karatsuba algorithm to work, as do the `b` and `d` variables.

Note that we use division and rounding down to calculate how much is half of the digits of `x` ❸. These mathematical operations are as complicated as multiplication and might not be available to the low-level hardware we are programming the Karatsuba algorithm for. In a real implementation, we could use another lookup table for these values: `HALF_TABLE = [0, 0, 1, 1, 2, 2, 3, 3...]`, and so on. Looking up `HALF_TABLE[n]` would evaluate to half of `n`, rounded down. An array of a mere 100 items would be sufficient for all but the most astronomical numbers and save our program from division and rounding. But our programs are for demonstration, so we'll just use the `/` operator and built-in rounding functions.

Once these variables are set up correctly, we can begin making the recursive function calls ❹. The first three steps involve recursive calls with arguments `a` and `b`, `c` and `d`, and finally `a + b` and `c + d`. The fourth step subtracts the results of the first three steps from each other ❺. The fifth step